Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
MoE, UGC & AICTE Approved
NAAC A++ Accredited

SentinelAI

Explainable machine learning for detecting malicious network traffic

23AI2001 AI for Cyber Security · Skill-Based Evaluation (III Internal)

Nitish (URK24CS5030) · Alston Renz (URK24CS5052) · Ferdinand Garson (URK24CS5068)
Fevin (URK24CS5067) · Jeffrey Joshua (URK24CS5065) · Venkat Gonuguntla (URK24CS5048)
Guide: Mr. Harivignesh K S, Assistant Professor



Too much network traffic to check by hand

- Most traffic is benign, but a small share is an attack: floods, scans, brute force, web attacks and bots.
- An intrusion detection system (IDS) flags the suspicious records so a security team can respond.
- Machine learning can learn attack patterns from labelled traffic, but analysts also need to know why a record was flagged.
- SentinelAI combines detection, attack-type naming and SHAP explanations in one local dashboard.

2.83 M network-flow records in CICIDS2017

78 numeric features per record

14 attack types, about 20% of records

Problem statement and objectives

Problem

Build a system that checks uploaded traffic data, predicts whether each record is benign or malicious, explains the prediction, and reports honestly how well it works on traffic it has never seen.

Analyze

Accept and validate CSV traffic data

Classify

Label each record BENIGN or MALICIOUS

Identify

Name the likely attack type, or say uncertain

Explain

Show which features drove a prediction

Evaluate

Compare models on familiar and unseen traffic

Record

Keep a local history of predictions

Literature review

Area	Key work	What SentinelAI takes from it
Intrusion detection	Denning (1987); Buczak & Guven (2016)	Separate normal from malicious activity
ML and DL for IDS	Liu & Lang (2019); Ahmad et al. (2021)	Wide range of models; supports a two-model comparison
Model foundations	Breiman (2001); Hosmer et al. (2013)	Basis for Random Forest and Logistic Regression
Datasets	Ring et al. (2019); Sharafaldin et al. (2018)	CICIDS2017's day-by-day capture enables holdout tests
Evaluation pitfalls	Sommer & Paxson (2010); Arp et al. (2022)	Random splits can make results look better than they are
Explainable AI	Ribeiro et al. (2016); Lundberg & Lee (2017)	SHAP explains individual predictions

Key gap: few student IDS projects test on a held-out day or explain their predictions.

Existing systems and their limitations



Signature-based IDS

Matches traffic against known attack patterns (e.g. Snort rules).

- Precise for known attacks
- Cannot recognise new attacks
- Rules need constant updates



Anomaly-based IDS

Models normal behaviour and flags deviations.

- Can flag unfamiliar behaviour
- Many false alarms
- Hard to say why an alert fired



Typical ML studies

Train a classifier on a labelled dataset and report accuracy.

- Very high reported accuracy
- Black-box predictions
- Usually tested on a random split only

Proposed system: SentinelAI

Two-stage detection

A binary model (Random Forest or Logistic Regression) flags attacks; a second Random Forest names the attack type.

Honest uncertainty

Below 0.60 confidence, or for attack types too rare to learn, the type is reported as uncertain instead of guessed.

SHAP explanations

For any record, the dashboard shows which features pushed the model toward attack or normal.

Unseen-day evaluation

Models are tested on a random split and on a whole held-out day (Wednesday).

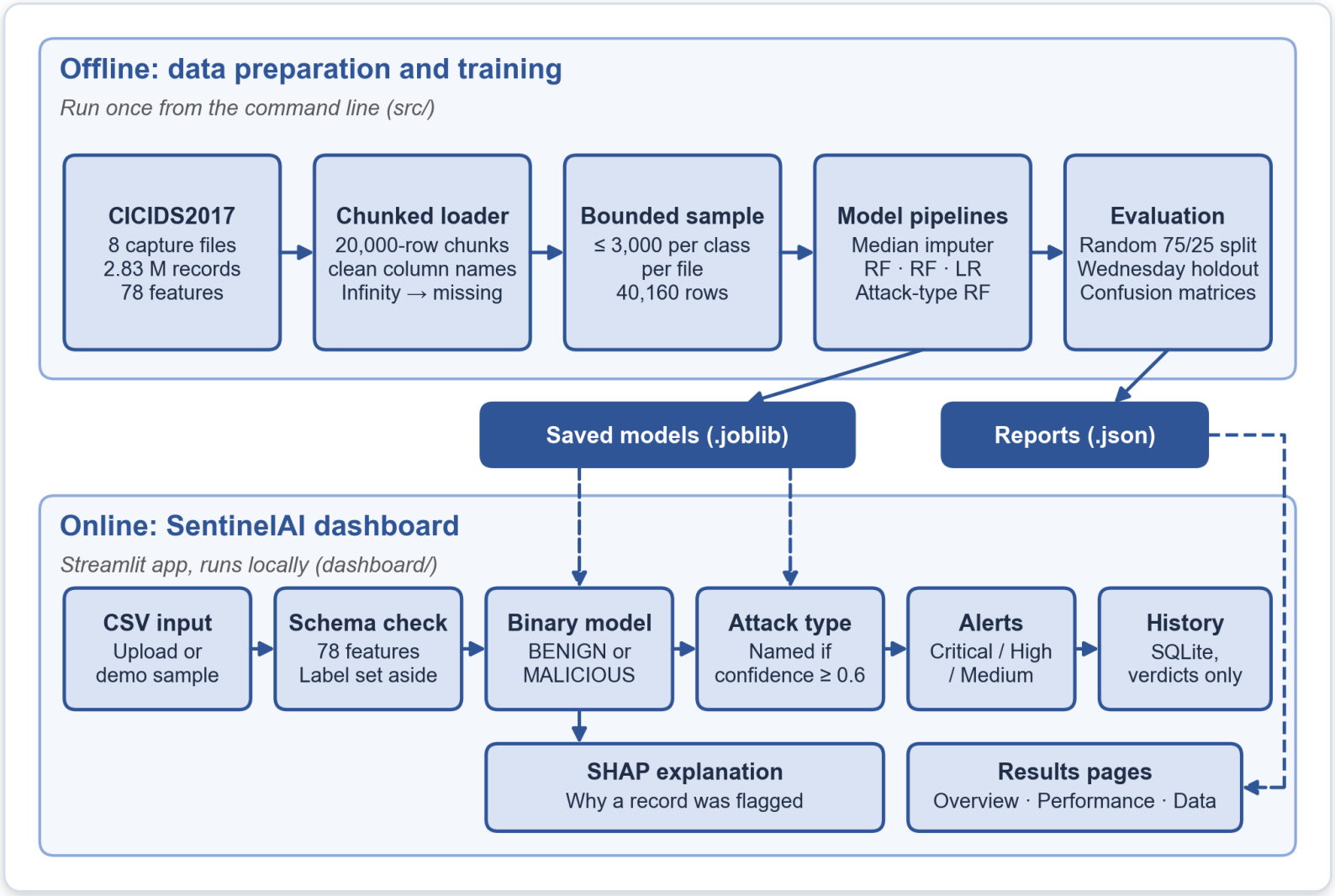
Local dashboard

A Streamlit app with six pages that runs offline; no data leaves the computer.

Memory-conscious pipeline

Chunked reading and a bounded sample keep training within 8 GB of RAM.

System architecture



Offline
Clean, sample and train once; save models and reports.

Online
The dashboard loads the saved models, checks each file, and returns alerts and explanations.

Methodology



Inspect

8 files, 2.83 M rows, schema and label counts

Clean

Standard column names; Infinity and NaN become missing

Sample

≤ 3,000 rows per class per file: 40,160 rows

Train

Median imputation fitted on training rows only

Evaluate

Random 75/25 split and Wednesday holdout

Explain

SHAP values per record and overall

No data leakage: every model is a scikit-learn pipeline, so imputation and scaling are learned from training rows only. Wednesday is removed from training entirely to test on genuinely new traffic, and Monday (benign only) cannot measure attack recall.

Implementation

Python · pandas

NumPy; chunked data pipeline

scikit-learn 1.7.2

Random Forest, Logistic Regression

SHAP

Tree and linear explainers

Streamlit · Plotly

Six-page local dashboard

SQLite

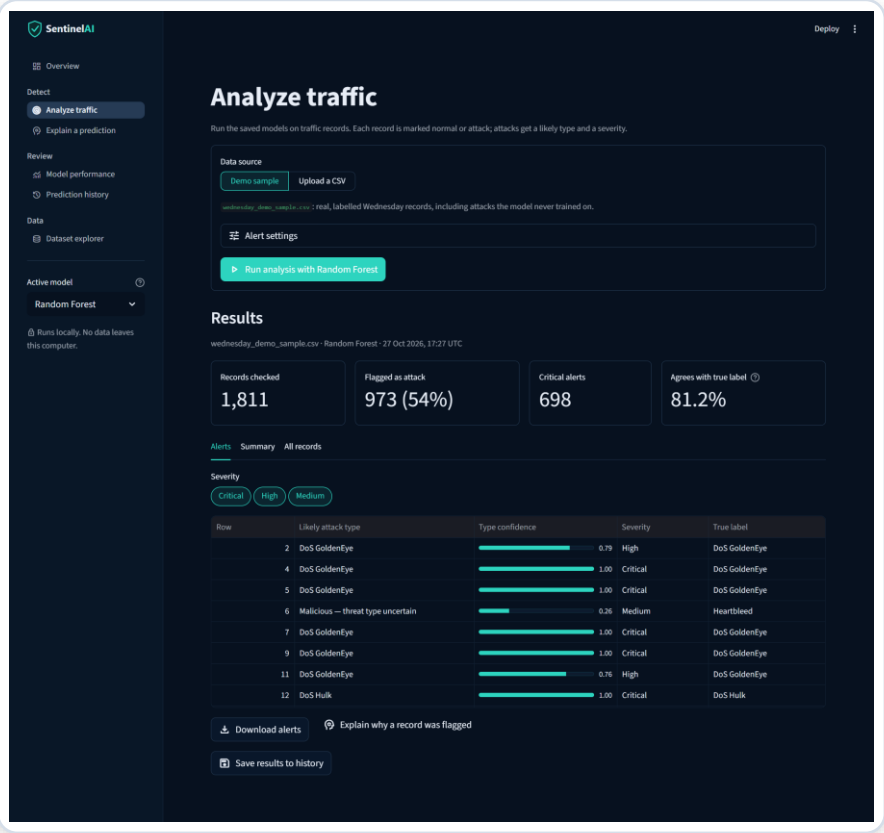
Prediction history (verdicts only)

pytest

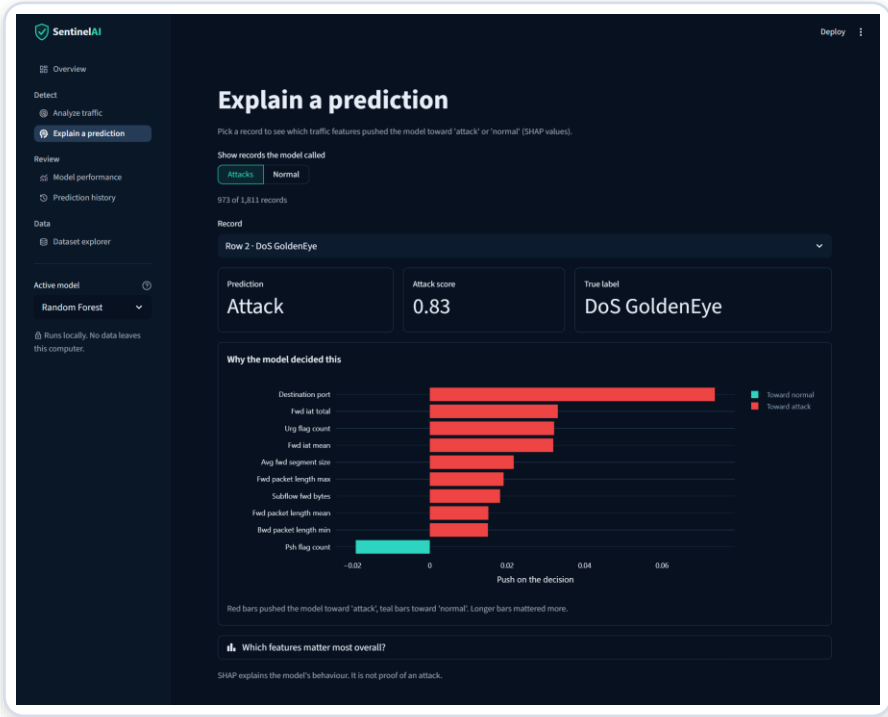
19 automated tests, all passing

Module	Role
src/data_pipeline.py	Streaming inspection and cleaning
src/train.py	Bounded sample, RF baseline, Wednesday holdout
src/compare_models.py	Three-model comparison, per-attack recall
src/threat_type.py	11-label attack-type model
dashboard/app.py	Streamlit pages and alert rules
dashboard/dashboard_utils.py	Upload validation, SHAP, SQLite

Results and output: the dashboard in action



Analyze traffic
Demo sample: 973 of 1,811 records flagged, 698 Critical; 81.2% agree with the true labels.



Explain a prediction
Row 2 (DoS GoldenEye): destination port and packet timing pushed toward attack.

6 pages

Overview

Analyze

Explain

Performance

History

Dataset

99% on familiar traffic, but only 20–33% on an unseen day

0.995

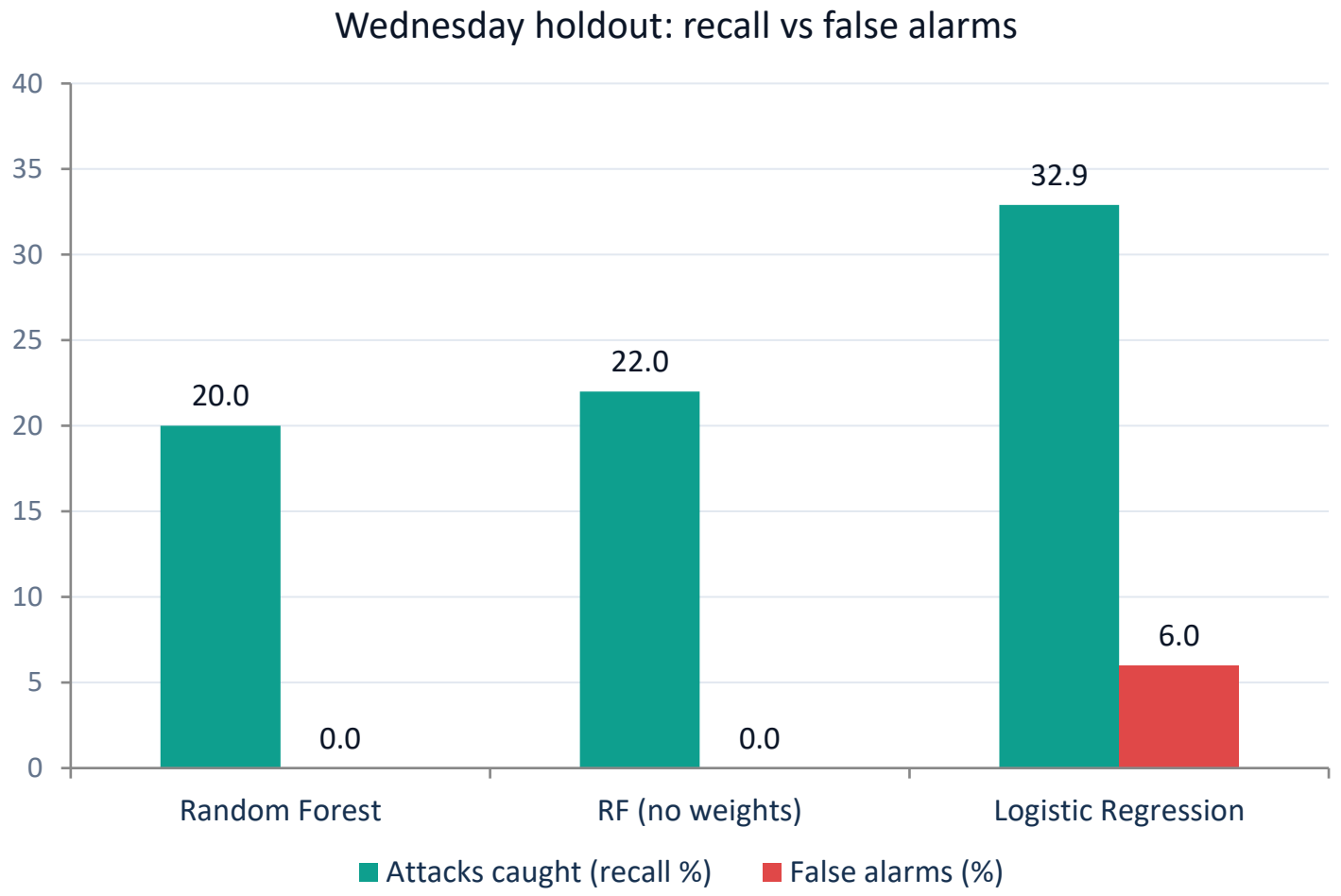
F1-score, random split (Random Forest)

20.0%

attacks caught on unseen Wednesday (RF)

32.9%

attacks caught on Wednesday (LR), with 6.0% false alarms



Conclusion

A working prototype

Detects, names and explains attacks in a local six-page dashboard.

Explainable by design

SHAP shows which traffic features drove every alert.

Honest about limits

Near-perfect on familiar data, but much weaker on an unseen day.

Main lesson

How a model is tested matters as much as the model itself.

Future scope



Validate on more held-out days

Leave out each day in turn to confirm and explain the drop in recall.



Tune thresholds and calibrate scores

Choose the alert threshold on validation data, not the test set.



Live and newer traffic

Add live capture and newer datasets such as CSE-CIC-IDS2018.



Stronger models

Try gradient boosting and deep learning against the same unseen-day test.



Reproducibility

Pin every dependency and add edge-case tests (malformed and oversized files).

References

1. D. E. Denning, "An Intrusion-Detection Model," IEEE Trans. Software Eng., 1987.
2. A. L. Buczak and E. Guven, "A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection," IEEE Comm. Surveys & Tutorials, 2016.
3. L. Breiman, "Random Forests," Machine Learning, vol. 45, 2001.
4. I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," ICISSP, 2018.
5. M. Ring et al., "A survey of network-based intrusion detection data sets," Computers & Security, 2019.
6. R. Sommer and V. Paxson, "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection," IEEE S&P, 2010.
7. D. Arp et al., "Dos and Don'ts of Machine Learning in Computer Security," USENIX Security, 2022.
8. M. T. Ribeiro, S. Singh and C. Guestrin, "Why Should I Trust You?: Explaining the Predictions of Any Classifier," KDD, 2016.
9. S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," NeurIPS, 2017.
10. S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," Nature Machine Intelligence, 2020.